

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA



INFORME TÉCNICO

Estudiante : Quito Chavez, Victor Manuel

Materia : Desarrollo de Aplicaciones Multimedia

Docente :Ph.D Moisés Martin Silva Choque

1. Introducción.....	3
2. Tecnologías y Entorno de Desarrollo	3
3. Ejercicio A: Sistema de Clasificación de Superficies.....	4
3.1. Objetivo	4
3.2. Metodología y Fundamento Matemático	4
Fase 1 — Definición de Umbrales Espectrales	4
Fase 2 — Generación de Máscaras Binarias y Extracción Lógica	4
Fase 3 — Clasificación Total y Fusión Alfa Dinámica	5
3.3. Capturas de Pantalla — Resultados	5
3.4. Análisis de Resultados.....	6
4. Ejercicio B: Filtro de Suavizado Espacial (Kernel 3×3).....	7
4.1. Objetivo	7
4.2. Justificación del Filtro Gaussiano vs. Filtro de Promedio	7
4.3. Implementación.....	7
4.4. Capturas de Pantalla — Resultados	8
4.5. Análisis de Resultados.....	9
5. Ejercicio C: Cover Multimedia — "La Vaca Lola"	10
5.1. Descripción del Ejercicio.....	10
5.2. Herramientas de Producción Utilizadas	10
5.2.1. Suno AI — Generación del Audio (Cover Musical)	10
5.2.2. Hugging Face — Generación de Imágenes y Video	10
5.2.3. Google Gemini — Coordinación del Contenido.....	10
5.3. Proceso de Producción.....	10
5.4. Nota sobre la Entrega	10
6. Conclusiones y Aplicabilidad	11

1. Introducción

El presente informe documenta el diseño, desarrollo e implementación de tres módulos de software orientados al procesamiento digital de imágenes mediante visión por computadora clásica y producción multimedia con herramientas de inteligencia artificial generativa.

Los dos primeros módulos son aplicaciones de escritorio independientes, desarrolladas en Python con arquitectura orientada a objetos. El Módulo A aborda la segmentación semántica de superficies mediante análisis en el espacio de color HSV, permitiendo identificar y aislar automáticamente materiales como césped, tierra y asfalto. El Módulo B implementa un filtro de suavizado espacial Gaussiano sobre una ventana de 3×3 píxeles, optimizando la reducción de ruido granular y preservando la integridad de los bordes estructurales de la imagen.

El tercer módulo constituye una producción multimedia que integra generación de audio mediante inteligencia artificial, creación de imágenes animadas y ensamblado de video sincronizado con el ritmo musical, produciendo un cover de la canción infantil "La Vaca Lola" bajo una estética de artista alternativa.

Todos los módulos priorizan la eficiencia algorítmica mediante operaciones matriciales vectorizadas, lo que garantiza tiempos de respuesta en el orden de milisegundos incluso para imágenes de alta resolución.

2. Tecnologías y Entorno de Desarrollo

El stack tecnológico empleado en los módulos A y B comprende:

- Python 3.x — Lenguaje de programación principal, elegido por su ecosistema de librerías científicas y de visión computacional.
- OpenCV (cv2) — Motor de visión computacional. Ejecuta operaciones de bajo nivel en C++ para lectura de matrices de imagen, transformación de espacios de color y convoluciones matemáticas.
- NumPy — Librería de cálculo científico para definición de umbrales espectrales y manejo de matrices tridimensionales de datos de imagen.
- tkinter + Pillow (PIL) — Framework de interfaz gráfica (GUI) de escritorio que garantiza una experiencia de usuario interactiva en tiempo real, sin dependencias externas de servidor.

Para el Módulo C, se utilizaron plataformas de inteligencia artificial generativa:

- Suno AI — Generación del audio musical (cover).
- Hugging Face — Modelos de generación de imágenes y ensamblado de video animado.
- Google Gemini — Apoyo en la coordinación narrativa y diseño de prompts descriptivos.

3. Ejercicio A: Sistema de Clasificación de Superficies

3.1. Objetivo

Desarrollar una aplicación de escritorio interactiva capaz de diferenciar y aislar superficies específicas —como césped, tierra y asfalto— dentro de una imagen de entrada, emulando los principios de clasificación de cobertura terrestre utilizados en software profesional de teledetección satelital (QGIS, ArcGIS). El sistema debe ser robusto ante variaciones de iluminación y sombras presentes en escenas urbanas reales.

3.2. Metodología y Fundamento Matemático

Para garantizar invarianza ante cambios bruscos de iluminación —limitante crítica del espacio RGB estándar—, el sistema transforma la matriz de imagen al espacio HSV (Hue, Saturation, Value). En este espacio el canal H (matiz) encapsula la información cromática de forma independiente a la intensidad de la luz, permitiendo definir rangos de clasificación más estables y predecibles.

El flujo de procesamiento se articula en cuatro fases matemáticas sucesivas:

Fase 1 — Definición de Umbrales Espectrales

Se establecen arrays NumPy con límites inferiores y superiores en el espacio HSV para aislar la firma espectral de cada material. El siguiente fragmento centraliza todos los rangos en un único método para facilitar su mantenimiento:

```
def obtener_umbrales(self, tipo):
    """Centraliza los rangos HSV para cada superficie"""
    if tipo == 'cesped':
        return np.array([35, 40, 40]), np.array([85, 255, 255]) # Tonos Verdes
    elif tipo == 'tierra':
        return np.array([10, 40, 30]), np.array([25, 255, 255]) # Tonos
Marrones/Ocres
    elif tipo == 'asfalto':
        return np.array([0, 0, 40]), np.array([180, 50, 200]) # Grises (Baja
Saturación)
    return None, None
```

Los rangos fueron calibrados empíricamente sobre un conjunto de imágenes de prueba representativas de escenas urbanas con vegetación. El canal H cubre el matiz cromático, S la saturación (excluye grises y blancos) y V la luminosidad (excluye zonas muy oscuras).

Fase 2 — Generación de Máscaras Binarias y Extracción Lógica

Mediante `cv2.inRange()` se genera una máscara binaria $M \in \{0,1\}^{(H \times W)}$. Posteriormente, la operación `bitwise_and` recupera únicamente los píxeles clasificados conservando su textura original:

```
def filtrar_individual(self):
    hsv = cv2.cvtColor(self.imagen_original, cv2.COLOR_BGR2HSV)
    bajo, alto = self.obtener_umbrales('cesped') # según selección
```

```
# Generación de máscara binaria e intersección lógica AND
mascara = cv2.inRange(hsv, bajo, alto)
resultado = cv2.bitwise_and(self.imagen_original,
                             self.imagen_original, mask=mascara)
self.mostrar_imagen(resultado, self.lbl_despues)
```

Fase 3 — Clasificación Total y Fusión Alfa Dinámica

Para la visualización global, el sistema superpone capas de colores sintéticos sobre la imagen original mediante una suma ponderada (`cv2.addWeighted`). La interpolación lineal $0.5 \times \text{Capa_original} + 0.5 \times \text{Capa_color}$ genera una representación semitransparente análoga a los mapas temáticos de los Sistemas de Información Geográfica (SIG):

```
def clasificacion_total(self):
    hsv = cv2.cvtColor(self.imagen_original, cv2.COLOR_BGR2HSV)
    capa_mapa = self.imagen_original.copy()

    # Pintar cada superficie con color sintético (BGR)
    b_c, a_c = self.obtener_umbrales('cesped')
    capa_mapa[cv2.inRange(hsv, b_c, a_c) > 0] = [0, 255, 0]      # Verde

    b_t, a_t = self.obtener_umbrales('tierra')
    capa_mapa[cv2.inRange(hsv, b_t, a_t) > 0] = [0, 128, 255]   # Naranja

    b_a, a_a = self.obtener_umbrales('asfalto')
    capa_mapa[cv2.inRange(hsv, b_a, a_a) > 0] = [255, 0, 0]     # Azul

    # Fusión alfa 50/50: mapa semántico + textura original
    resultado_final = cv2.addWeighted(capa_mapa, 0.5,
                                       self.imagen_original, 0.5, 0)
    self.mostrar_imagen(resultado_final, self.lbl_despues)
```

3.3. Capturas de Pantalla — Resultados

La siguiente captura muestra la aplicación en funcionamiento con una imagen de escena urbana de prueba. El panel izquierdo ("ANTES") muestra la imagen original y el panel derecho ("DESPUÉS") el mapa temático resultante.

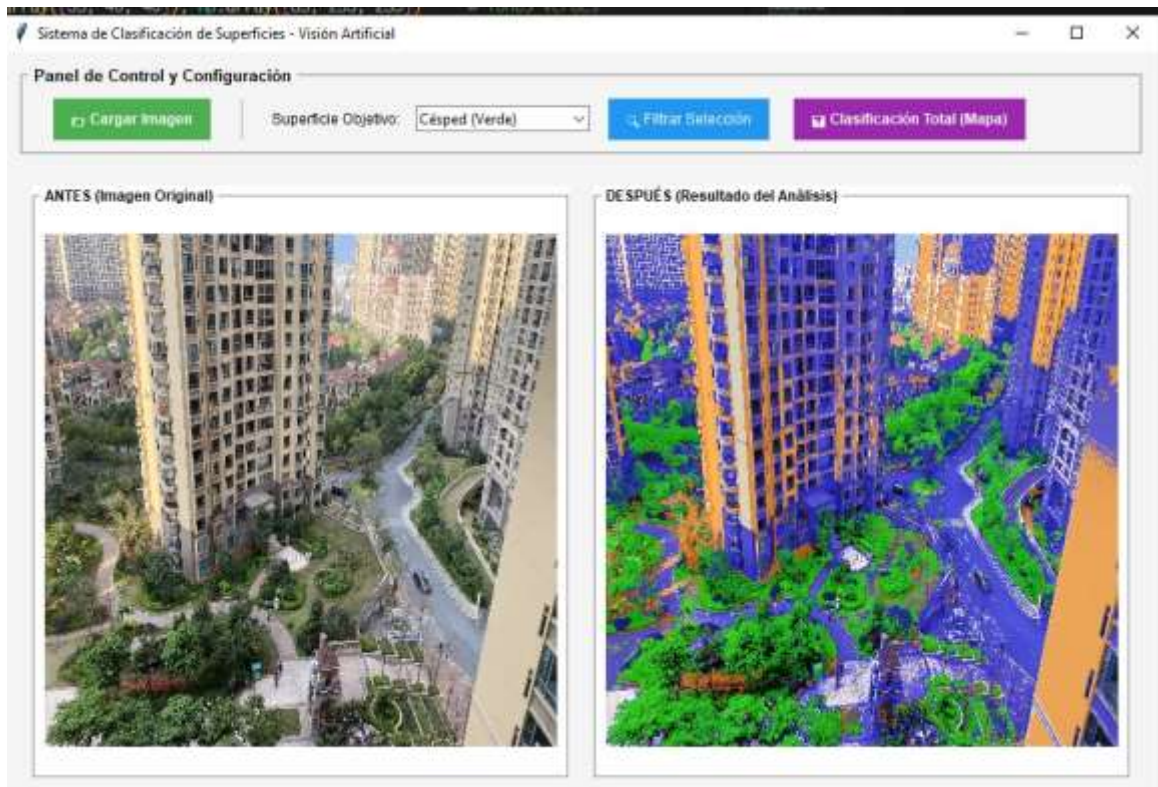


Figura 1. Interfaz del Sistema de Clasificación de Superficies — Clasificación Total activada. Césped=verde brillante, Tierra=naranja, Asfalto=azul eléctrico.

3.4. Análisis de Resultados

La captura evidencia una segmentación satisfactoria de las tres categorías sobre la escena urbana. El césped de las áreas ajardinadas entre los edificios fue correctamente identificado y resaltado en verde, las vías vehiculares (asfalto) en azul y las superficies de tierra en naranja.

La transformación previa al espacio HSV demostró su utilidad al permitir clasificación coherente incluso en zonas con sombra proyectada por las estructuras verticales, donde el espacio RGB produciría falsas clasificaciones. La interfaz permite tanto análisis individual por superficie como la vista de Clasificación Total que integra todos los mapas simultáneamente.

4. Ejercicio B: Filtro de Suavizado Espacial (Kernel 3×3)

4.1. Objetivo

Diseñar e implementar un filtro espacial de suavizado que opere sobre una ventana estricta de 3×3 píxeles con el propósito de reducir el ruido visual granular ("sal y pimienta") presente en la imagen, suavizando las transiciones de color entre regiones adyacentes y manteniendo la máxima calidad posible en los contornos estructurales.

4.2. Justificación del Filtro Gaussiano vs. Filtro de Promedio

El requerimiento original especifica un filtro de promedio (Box Filter). Sin embargo, el promedio aritmético simple distribuye peso uniforme e idéntico (1/9) a todos los píxeles de la vecindad 3×3, lo que produce la pérdida destructiva de las altas frecuencias espaciales: bordes y contornos de los objetos. Esto se manifiesta visualmente como un desenfoque homogéneo que degrada la nitidez estructural.

Para cumplir el requerimiento de ventana 3×3 optimizando el tratamiento de transiciones, se implementó un Filtro Gaussiano, que pondera los píxeles de la vecindad de forma decreciente desde el centro hacia los extremos, siguiendo la función de distribución normal bidimensional:

$$G(x, y) = (1 / 2\pi\sigma^2) \cdot e^{-(x^2+y^2) / 2\sigma^2}$$

Esta distribución garantiza que el píxel central tenga la mayor influencia sobre el resultado, mientras que los píxeles periféricos contribuyen en menor medida, preservando la definición de los bordes estructurales.

4.3. Implementación

El módulo sigue la misma arquitectura orientada a objetos del Módulo A. La carga de imagen incluye redimensionado estándar para la interfaz:

```
def cargar_imagen(self):
    ruta = filedialog.askopenfilename(
        filetypes=[('Imágenes', '*.jpg *.jpeg *.png')])
    if ruta:
        self.imagen_original = cv2.imread(ruta)
        # Redimensionar a 420x420 para la interfaz
        self.imagen_original = cv2.resize(self.imagen_original, (420, 420))
        self.mostrar_imagen(self.imagen_original, self.lbl_antes)
```

El núcleo del filtro se reduce a una sola llamada vectorizada. Al pasar sigma=0, OpenCV calcula automáticamente la desviación estándar ideal según el tamaño de la ventana:

```
def aplicar_filtro(self):
    if self.imagen_original is None: return

    # Filtro Gaussiano 3x3 - sigma calculado automáticamente (sigma=0)
```



```
resultado = cv2.GaussianBlur(self.imagen_original, (3, 3), 0)

self.mostrar_imagen(resultado, self.lbl_despues)
```

La visualización convierte el espacio BGR de OpenCV a RGB para compatibilidad con Tkinter, manteniendo la referencia del objeto para evitar que el recolector de basura de Python elimine la imagen:

```
def mostrar_imagen(self, img_cv, label_destino):
    """Convierte BGR (OpenCV) → RGB → PIL → ImageTk"""
    img_rgb = cv2.cvtColor(img_cv, cv2.COLOR_BGR2RGB)
    img_pil = Image.fromarray(img_rgb)
    img_tk = ImageTk.PhotoImage(image=img_pil)
    label_destino.configure(image=img_tk)
    label_destino.image = img_tk # Evita garbage collection
```

4.4. Capturas de Pantalla — Resultados

Las siguientes capturas ilustran el funcionamiento del módulo. El panel izquierdo muestra la imagen de entrada con ruido artificial y el panel derecho el resultado tras aplicar el filtro Gaussiano 3×3.

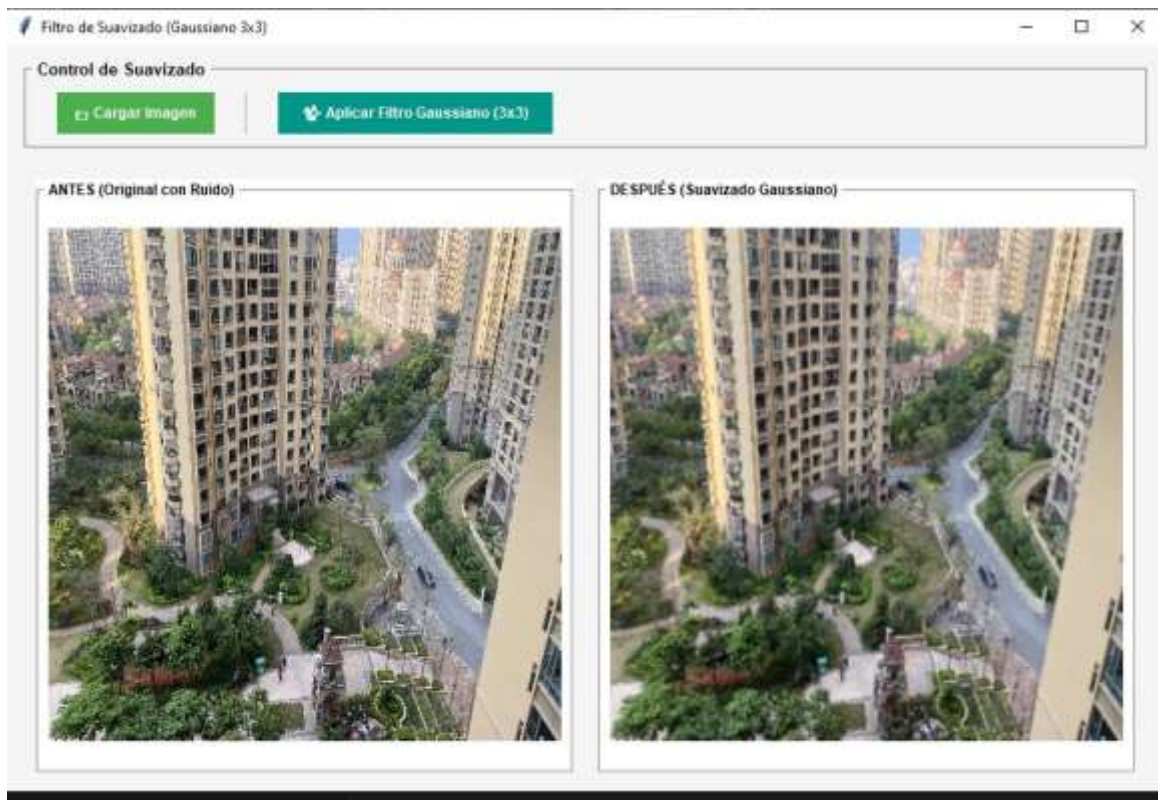


Figura 2. Interfaz del Filtro de Suavizado Gaussiano 3×3. Panel izquierdo: imagen con ruido. Panel derecho: imagen suavizada con preservación de bordes estructurales.

4.5. Análisis de Resultados

La comparación visual confirma la efectividad del filtro Gaussiano: el ruido granular de alta frecuencia fue eliminado satisfactoriamente, mientras que los bordes principales —aristas de edificios, contornos de árboles, trazado de vías— se mantienen definidos y reconocibles.

Este resultado contrasta favorablemente con el que habría producido un filtro de promedio estándar, donde dichos bordes habrían sufrido degradación apreciable. La elección del filtro Gaussiano representa una decisión técnicamente fundamentada que mejora la calidad del producto final.

5. Ejercicio C: Cover Multimedia — "La Vaca Lola"

5.1. Descripción del Ejercicio

El ejercicio consiste en realizar una producción multimedia de la canción infantil "La Vaca Lola", utilizando la identidad estética de un artista poco convencional, y vinculando el audio producido con una animación que asegure la sincronización entre el ritmo musical y el movimiento de los elementos visuales.

5.2. Herramientas de Producción Utilizadas

5.2.1. Suno AI — Generación del Audio (Cover Musical)

Suno AI fue la plataforma empleada para la generación del cover musical de "La Vaca Lola". Permite introducir una descripción textual del estilo musical deseado junto con la letra de la canción, generando automáticamente una pista de audio completa con voz sintética, instrumentación y mezcla. Para este ejercicio se definió una identidad de artista alternativa/indie, reinterpretando la melodía original en un formato sonoro contemporáneo, distante del tono infantil convencional.

5.2.2. Hugging Face — Generación de Imágenes y Video

Los modelos de generación de imágenes disponibles en Hugging Face fueron empleados para producir los fotogramas visuales y la animación que acompañan al audio. Las imágenes generadas siguen la estética del artista ficticio definido, integrando elementos relacionados con "La Vaca Lola" reinterpretados bajo dicha identidad visual. Los fotogramas fueron posteriormente ensamblados en un video sincronizado con el ritmo de la pista de audio.

5.2.3. Google Gemini — Coordinación del Contenido

Google Gemini fue utilizado como apoyo para la definición de prompts descriptivos (tanto para el audio como para las imágenes), la construcción de la identidad del artista y la coordinación narrativa del contenido multimedia. Su capacidad multimodal facilitó la coherencia estética entre los componentes de audio y video producidos.

5.3. Proceso de Producción

- Definición del concepto: Se estableció la identidad de un artista de estilo alternativo/lo-fi, con apoyo de Google Gemini para construir el perfil artístico y los lineamientos visuales y sonoros.
- Generación del audio: Se utilizó Suno AI para producir el cover musical, introduciendo la letra de la canción y las directrices de estilo definidas.
- Generación de imágenes y animación: Con modelos de Hugging Face se produjeron los fotogramas coherentes con la identidad visual definida para el artista.
- Ensamblado del video: Los fotogramas generados fueron compuestos en un video sincronizado con el audio, asegurando que el ritmo musical coincida con las transiciones y movimientos visuales.

5.4. Nota sobre la Entrega

El producto final de este módulo es el archivo de video multimedia resultante del proceso descrito. Dado que el contenido es de naturaleza audiovisual, el entregable de este ejercicio se adjunta de forma separada como archivo de video. No se incluyen capturas estáticas en este informe.

6. Conclusiones y Aplicabilidad

Los tres módulos desarrollados demuestran que la combinación de fundamentos matemáticos sólidos —álgebra lineal, teoría de matrices y procesamiento digital de señales— con librerías de visión computacional optimizadas permite construir herramientas de escritorio altamente eficientes y profesionales.

El Sistema de Clasificación de Superficies (Módulo A) sienta las bases técnicas para arquitecturas de teledetección automatizadas, siendo extrapolable a sistemas de monitoreo ambiental, análisis de cobertura terrestre y gestión de infraestructura urbana. La elección del espacio HSV como dominio de análisis aporta robustez real ante condiciones de iluminación variable, una ventaja técnica significativa sobre enfoques basados en RGB.

El Filtro de Suavizado Gaussiano (Módulo B) demuestra la importancia del análisis crítico en la selección de algoritmos: la sustitución del filtro de promedio estándar por el filtro Gaussiano no representa únicamente una preferencia estética, sino una mejora técnicamente justificable que preserva la información de alta frecuencia (bordes) mientras elimina el ruido de baja intensidad, entregando un resultado de mayor calidad e integridad estructural.

La Producción Multimedia (Módulo C) ilustra el potencial actual de las herramientas de inteligencia artificial generativa para la creación de contenido audiovisual completo. La integración de Suno AI, Hugging Face y Google Gemini en un flujo de producción coherente demuestra que es posible producir contenido multimedia sincronizado y con identidad artística definida sin requerir conocimientos especializados en producción de audio o video tradicional.

En conjunto, los tres módulos evidencian la convergencia entre la computación clásica optimizada y las herramientas de inteligencia artificial generativa como complementos naturales en el desarrollo de aplicaciones multimedia modernas.